```
    | aft selected.</div>';document.getElementById("route").innerHTML='<div class="empty">Route details appear on selection.</
    | div>';chart.data.labels=[];chart.data.datasets[0].data=[];chart.update("none")}}}
275     function select(i){const l=state.layers.get(i);if(!l)return;state.sel=i;const a=l.data;map.setView([a.latDD,a.lonDD],6
    | );l.m.setZIndexOffset(1000);l.m.openPopup();panels(a);drawRoute(a);openTab("flight");filterAll()}
276     function renderList(){const q=els.search.value.trim().toUpperCase();const arr=state.rows.filter(a=>!q||a.callsign.incl
    | udes(q)||a.icaoAddress.includes(q)).sort((a,b)=>a.callsign.localeCompare(b.callsign));if(!arr.length){els.list.innerHTML
    | ='<li class="empty">No flights match current search.</li>';return}els.list.innerHTML="";arr.forEach(a=>{const li=documen
    | t.createElement("li");li.className=`row${state.sel===a.icaoAddress?" active":""}`;li.innerHTML=`<div class="call">${a.ca
    | llsign}${a.isPredicted?' <span style="color:#ff7d7d">(PRED)</span>':''}</div><div class="meta">${a.icaoAddress} | ALT ${
    | Math.round(a.altitudeMM/1000)}m | SPD ${Math.round(a.horVelocityCMS/100)}m/s${a.isPredicted?` | AGE ${Math.round(a.predi
    | ctionAgeSec)}s`:''}</div>`;li.addEventListener("click",()=>select(a.icaoAddress));els.list.appendChild(li)})}
277     function filterAll(){const q=els.search.value.trim().toUpperCase();let vis=0;for(const [i,l] of state.layers){const a=
    | l.data,m=!q||a.callsign.includes(q)||a.icaoAddress.includes(q)||state.sel===i;showLayer(l,m);if(m)vis++}els.visible.text
    | Content=String(vis);renderList()}
278     // ---- mode/status badge ------------------------------------------------
279     const modeBadge=document.getElementById("mode-badge");
280     const modeToggle=document.getElementById("mode-toggle");
281     const liveBanner=document.getElementById("live-banner");
282     let _lastStatus=null;
283     async function refreshStatus(){
284       try{
285         const r=await fetch("/api/status",{cache:"no-store"});
286         if(!r.ok)return;
287         const s=await r.json();
288         _lastStatus=s;
289         const simulationOn=s.mode==="simulation";
290         modeToggle.textContent=`Simulation: ${simulationOn?"TRUE":"FALSE"}`;
291         modeToggle.classList.toggle("sim-on",simulationOn);
292         if(s.mode==="simulation"){
293           modeBadge.textContent="? SIM";
294           modeBadge.className="sim";
295           liveBanner.style.display="none";
296         } else {
297           // live mode
298           if(s.connected){
299             const ago=s.lastDataSecondsAgo!=null?` · data ${s.lastDataSecondsAgo}s ago`:"";
300             modeBadge.textContent=`? LIVE${ago}`;
301             modeBadge.className="live-ok";
302             liveBanner.style.display="none";
303           } else if(s.bindError){
304             modeBadge.textContent="? LIVE ERR";
305             modeBadge.className="live-err";
306             liveBanner.style.display="block";
307             liveBanner.innerHTML=`<b>? PingStation socket error</b><br>${s.bindError}<br>Check that port ${s.listenPort} i
    | s not in use by another program.`;
308           } else {
309             modeBadge.textContent="? LIVE WAIT";
310             modeBadge.className="live-wait";
311             liveBanner.style.display="block";
312             liveBanner.innerHTML=`<b>Waiting for PingStation connection?</b><br>
313               Listening on port <b>${s.listenPort}</b>.  
314               Configure the PingStation device to send its RAW stream to
315               <b>this PC's IP address : ${s.listenPort}</b> (TCP).<br>
316               Frames decoded so far: ${s.totalFramesDecoded}  | 
317               Connections received: ${s.totalConnections}<br>
318               <span style="font-size:10px;color:#9dbdde">Config file: ${s.configFile}</span>`;
319           }
320         }
321       }catch(e){/* ignore status errors */}
322     }
323
324     // ---- aircraft data refresh --------------------------------------------
325     async function refresh(){
326       try{
327         const r=await fetch("/api/aircraft",{cache:"no-store"});
328         if(!r.ok)throw new Error(r.status);
329         const p=await r.json(),src=Array.isArray(p)?p:(p.aircraft||[]),arr=src.map(norm).filter(a=>Number.isFinite(a.latDD
    | )&&Number.isFinite(a.lonDD));
330         state.rows=arr;
331         els.tracked.textContent=String(arr.length);
332         // Show "Live" / "Sim" in the Feed chip based on mode
333         if(_lastStatus&&_lastStatus.mode==="simulation"){
334           els.status.textContent="Sim";
335           els.status.style.color="#8ecae6";
336         } else {
337           els.status.textContent=arr.length?"Live":"Live";
338           els.status.style.color="var(--ok)";
339         }
340         const active=new Set();
341         arr.forEach(a=>{active.add(a.icaoAddress);ensure(a);if(state.sel===a.icaoAddress){panels(a);drawRoute(a)}});
342         cleanup(active);
343         filterAll();
344       }catch(e){
345         els.status.textContent="Error";
346         els.status.style.color="var(--bad)";
347       }
348     }
349
350     document.getElementById("zin").addEventListener("click",()=>map.zoomIn());
351     document.getElementById("zout").addEventListener("click",()=>map.zoomOut());
352     document.getElementById("center").addEventListener("click",()=>map.setView([22.2,79.2],5));
353     document.querySelectorAll("#tabs button").forEach(b=>b.addEventListener("click",()=>openTab(b.dataset.tab)));
354     els.search.addEventListener("input",filterAll);
355     modeToggle.addEventListener("click",async()=>{
356       const simulation=!(_lastStatus&&_lastStatus.mode==="simulation");
357       modeToggle.disabled=true;
358       modeToggle.textContent="Switching...";
359       try{
360         const r=await fetch("/api/mode",{method:"POST",headers:{"Content-Type":"application/json"},body:JSON.stringify({si
    | mulation})});
361         const response=await r.json().catch(()=>({}));
362         if(!r.ok)throw new Error(response.error||`HTTP ${r.status}`);
363         _lastStatus=response;
364         await refreshStatus();
365         await refresh();
366       }catch(e){
367         modeToggle.textContent=`Switch failed: ${e.message||"unknown error"}`;
368         setTimeout(refreshStatus,2500);
369       }finally{
370         modeToggle.disabled=false;
404     refreshStatus();
405     refresh();
406     setInterval(refresh,1000);
407     setInterval(refreshStatus,4000);
408   </script>
```